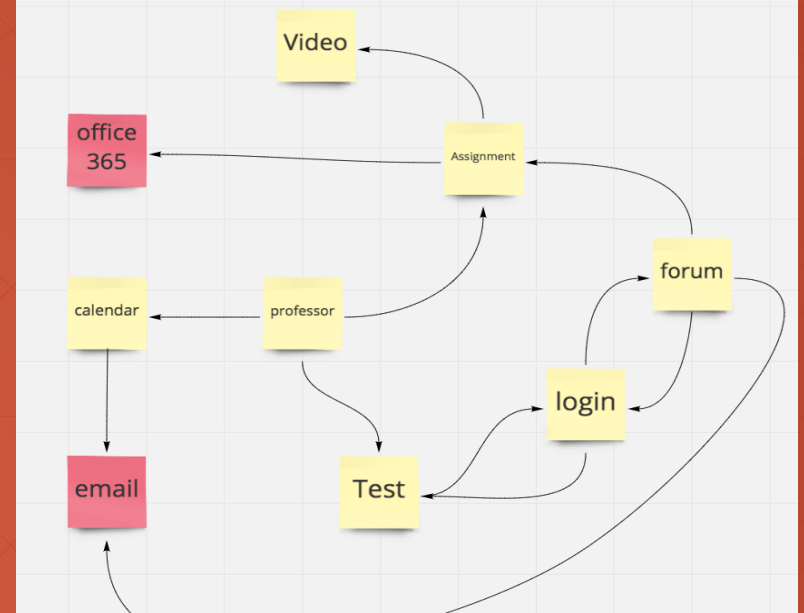


抽象类、接口与面向对象建模方法

北京理工大学计算机学院
金旭亮



1 抽象类



抽象类和抽象方法

- 有abstract修饰的类称为“**抽象类**”，它只定义了什么方法应该存在，不能直接创建对象。你必须从它派生出一个子类，并在子类中实现其未实现的方法之后，才能使用new关键字创建它的实例。
- 在方法前加上abstract就形成**抽象方法**，这种类型的方法，只有方法声明，没有实现代码。

抽象类、抽象方法实例

```
abstract class Person {  
    public abstract String getDescription();  
    // 其他成员（包括普通方法和字段）  
}
```



一个抽象类中可以包含非抽象方法，也可以象普通类一样，定义字段。



包含抽象方法的类一定是抽象类，但抽象类中的方法不一定是抽象方法。

抽象类的三种“类型”



1. 直接定义了一个抽象方法
2. 继承了一个抽象父类，但没有完全实现父类包含的抽象方法
3. 实现了一个接口，但没有完全实现此接口所包容的抽象方法。

从抽象类继承

- 从抽象类继承的子类必须实现父类的所有抽象方法，否则，它仍然是抽象类。

```
class Employee extends Person {  
    public String getDescription() {  
        //Do Something...  
        return "";  
    }  
}
```

使用抽象类

- 抽象类不能创建对象，一般用它来引用子类对象。
- 实例：

```
Person p = new Employee();
```

以下模式总是成立的

```
抽象类 抽象类变量 = new 派生自抽象类的具体子类();
```

抽象类的应用实例

在现实生活中，属于同一种类的事物，在特定的方面，表现出不同的特点，比如狗和猫都会叫，但两者的叫声是不同的。

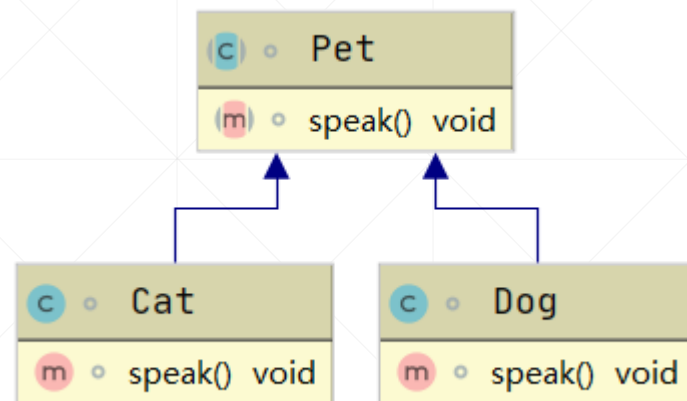
请使用面向对象的观点，为其抽象建模。



“狗与猫”的面向对象建模思路

如果为狗和猫建模，可以这样设计：

- 设计一个Dog类，代表狗，设计一个Cat类，代表猫。
- 由于狗和猫都是一种（IS_A）宠物，所以设计一个Pet类，让Dog和Cat成为它的子类。
- 由于狗和猫都会叫，所以给Pet类添加一个speak()方法，又由于并不存在一种“统一”的宠物叫声，而狗和猫的叫声又不一样，所以，应该让Pet.speak()方法成为抽象的。Pet作为抽象类，而Cat和Dog重写基类的speak()方法。



为“狗和猫”建立的面向对象模型

宠物基类

```
abstract class Pet {  
    abstract void speak();  
}
```



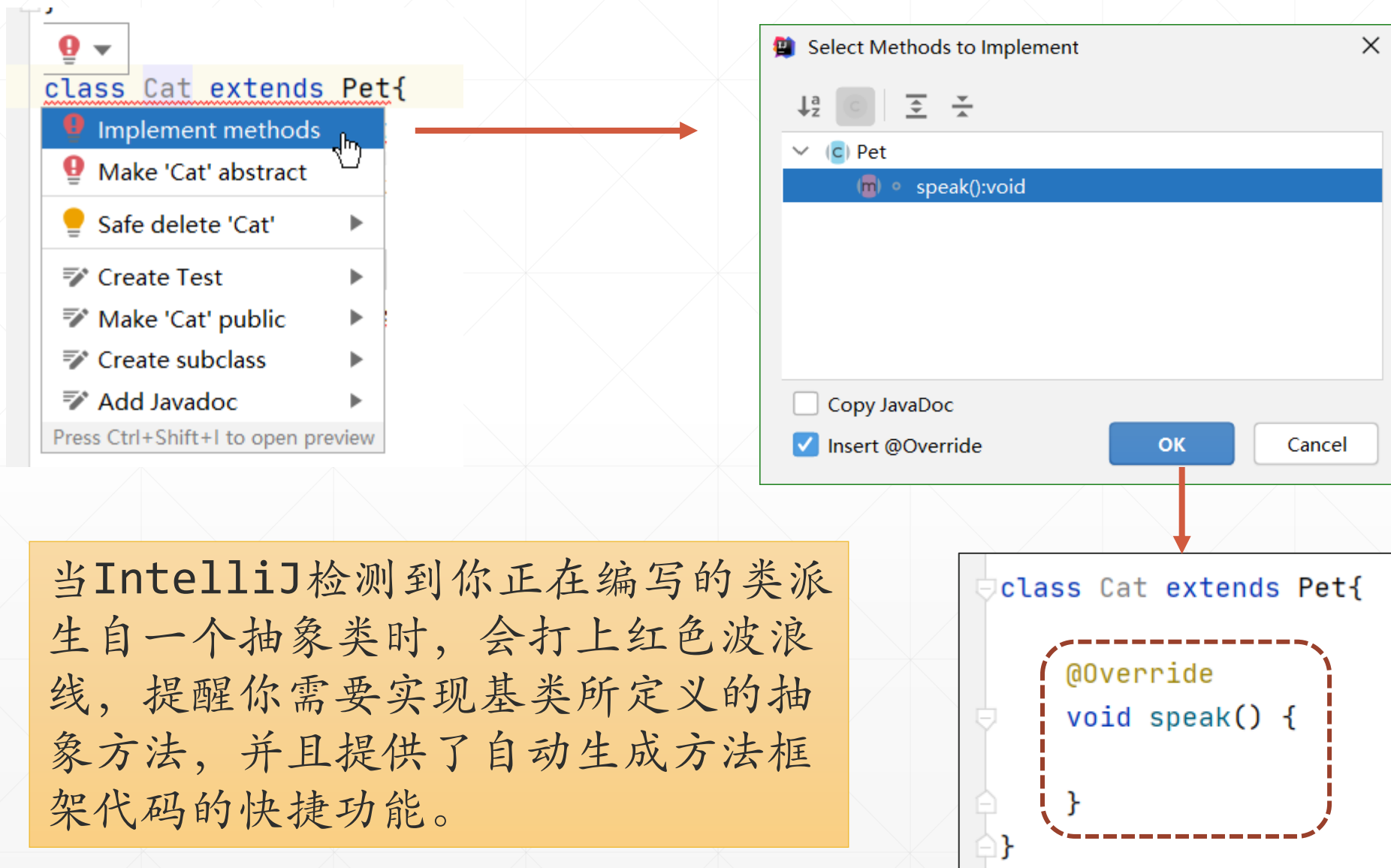
```
public static void main(String[] args) {  
    Pet d = new Dog();  
    //汪汪汪~~~  
    d.speak();  
    //喵喵喵~~~  
    Pet c = new Cat();  
    c.speak();  
}
```

```
class Dog extends Pet {  
  
    @Override  
    void speak() {  
        System.out.println("汪汪汪~~~");  
    }  
}
```

```
class Cat extends Pet {  
  
    @Override  
    void speak() {  
        System.out.println("喵喵喵~~~");  
    }  
}
```

示例项目：UseAbstractClass

多学一招：IntelliJ提供的快捷功能



当IntelliJ检测到你正在编写的类派生自一个抽象类时，会打上红色波浪线，提醒你需要实现基类所定义的抽象方法，并且提供了自动生成方法框架代码的快捷功能。

小结



只要客观事物之间是“IS_A”关系，就可以使用“继承”来为之建立模型。



事物的共性，放在基类中，定义为基类的字段或方法。如果这些“共性”是本类事物都一样的，实现为普通的基类方法，比如宠物都有主人，如果是“大同小异”，比如猫和狗叫声不一样，则在基类中定义抽象方法，在子类中为这些抽象方法提供不同的实现。

2 接口



技术背景

先来看一个现实生活中的实例：

鸭子是一种鸟，会游泳，同时又是一种食物。

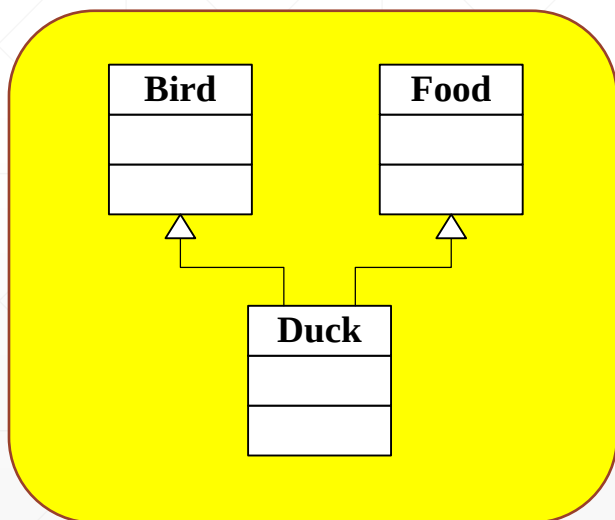


- 如何使用面向对象的思想，为上述场景建立一个系统模型（建模）？
- 上面的描述中，什么叫作“建立一个系统模型”？

所谓“建模”，就是为现实世界创建一个对应的、可以使用计算机实现的模型。具体到本例中，就是：你打算设计哪些类来“仿真”鸭子这一现实事物？

一种面向对象的设计方案

- 因为“鸭子是一种 (**IS_A**) 鸟，又是一种 (**IS_A**) 食物”，所以我们可以创建三个类：Bird代表鸟，Food代表食物，Duck代表鸭子，让Duck派生自Bird和Food。



这个方案存在的问题：

- (1) Java不支持多继承
- (2) “会游泳”这个方法放在哪个类中？放到Bird中似乎并不合适，因为只有部分鸟类会游泳。放到Duck中也不好，因为会游泳的水鸟不止鸭子一种。

换一个思路.....

能否把“会游泳”、“是一种食物，可以吃”这种特性独立出来作为一种“可选项”，需要时可以“附加”到具体对象上？

如果真能实现的话，这样一来，水鸟可以拥有“会游泳”这个特性，其它种类的鸟则没有这个特性（但它可能有其他的特性），从而完美地解决问题。

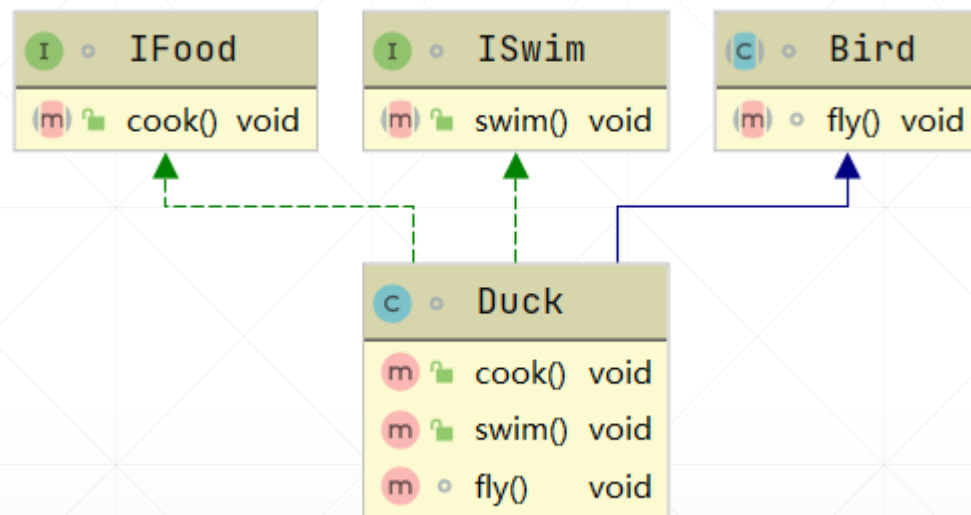


在面向对象领域中，“**接口 (interface)**”被用来抽象对象的行为特性或外部特征。

使用“接口”解决鸭子问题

Duck类派生自Bird类，实现IFood和ISwim接口，这个方案很清楚地表明了：

鸭子是一种鸟（用“继承”特性表达），会游泳，同时又是一种食物（用它所实现的“接口”表达）。



示例：DuckAndBird.java

Java中“接口”的语法特性



定义一个接口，采用关键字`interface`；某个类实现某个接口，采用关键字`implements`。



接口的成员函数自动成为`public`的，数据成员自动成为`static`和`final`的。



如果接口不声明为`public`的，则自动变为`package`的。



一个类可以同时实现多个接口，但只能有一个基类。基类必须写在第一位，后面跟着的是它所实现的多个接口。

```
class Duck extends Bird
    implements ISwim, IFood {

    @Override
    public void cook() {...}

    @Override
    public void swim() {...}

    @Override
    void fly() {...}
}
```

接口的使用



接口类型 接口类型的变量 = new 实现了接口的具体类型 () ；

```
public class DuckAndBird {  
    public static void main(String[] args) {  
        Duck duck = new Duck();  
        duck.fly();  
        //Duck对象实现了IFood和ISwim接口,  
        //所以可以赋值给相应的变量  
        IFood food = duck;  
        food.cook();  
        ISwim swimAnimal = duck;  
        swimAnimal.swim();  
    }  
}
```

接口的扩充

- 可以通过继承接口来扩充已有接口，并形成一个新的接口。

```
interface OneInterface {  
    void f1();  
}  
  
interface TwoInterface extends OneInterface {  
    void f2();  
}
```



实现子接口的类，必须实现“父”与“子”接口所定义的所有方法，才能被实例化（即new出一个对象）。

利用接口定义常量

```
public interface ArrayBound {  
    public static final int LOWBOUND = 1;  
    public static final int UPBOUND = 100;  
}
```

以这种方式定义的常量，说明这些常量需要被实现这个接口的类所使用。请注意要为它们设计出有意义的名字，以便保证代码的可读性。

- 只要一个类声明实现了这个接口，就可以直接使用这些常量。
- 在实际开发中，这种编程方式非常常见。



定义在接口中的常量必须被初始化

函数式接口

JDK 8中引入了一种“函数式”接口，这种接口只定义有一个公有方法，使用`@FunctionalInterface`注解加以标注：

```
@FunctionalInterface
public interface MyFunctionInterface {
    void process();
}
```



这种类型的接口，主要用于“函数式编程”场景，使用它定义的变量，可以接收一个Lambda表达式。

函数式接口示例

```
public class UseFunctionalInterface {  
    public static void main(String[] args) {  
        MyFunctionInterface obj = () -> {  
            var info :String = ""  
                |工作完成于: %s  
                |"".formatted(LocalDate.now());  
            System.out.println(info);  
        };  
        obj.process();  
    }  
}
```

函数式接口变量，可以引用一个Lambda表达式。

有关Lambda表达式的内容，后面课程介绍。

示例运行结果：

工作完成于：2020-10-02

JDK9引入的新特性-1

```
public interface MyInterface {  
    //JDK 9: 接口中可以定义私有成员，它可以被接口的默认方法调用  
    private void secret() {  
        System.out.println("Secret Method defined in interface");  
    }  
    //接口的默认方法  
    default void defaultMethod(){  
        secret();  
        System.out.println("defaultMethod()");  
    }  
    void func();  
}
```



接口中的私有方法，主要用于封装一些接口的默认方法（可以有多个）需要调用的公用代码。

JDK9引入的新特性-2

```
public class MyClass implements MyInterface {  
    @Override  
    public void func() {  
        System.out.println("MyClass.func()");  
    }  
  
    public static void main(String[] args) {  
        MyInterface obj = new MyClass();  
        obj.defaultMethod();  
        obj.func();  
    }  
}
```

直接使用接口所定义的默认方法

实现了接口的具体类型，如果没有重写，就直接“继承”接口所定义的默认方法。



Run: MyClass x

```
"C:\Program Files\Java\jdk-15\bin\java.exe"  
Secret Method defined in interface  
defaultMethod()  
MyClass.func()  
  
Process finished with exit code 0
```

接口与回调



在实际开发中，接口有着广泛的应用，一种非常典型的场景就是利用它来实现“**回调 (callback)**”这种编程技巧。



所谓“回调 (callback)”，简单地说，就是你准备好一个方法，通知别人：“我这里有一个方法提供给你，你可以在合适的时间自己去调用它，我就不管了”。

传统的编程方式，是你去“调用”某个方法，是“**主动的**”，而回调，则是你事先准备好一个方法，让别人来调用，别人什么时候调用这个方法，是否调用这个方法，你是不知道的，整个过程是“**被动**”的，这就是“回调”的含义。

基于接口的“回调”实例-1

Comparator<T>是JDK中用于定义两个对象大小比较规则的一个接口。

以下定义了一个类，实现了这个接口：

```
//按字符串长度决定两个字符串“谁大谁小”  
class LengthComparator implements Comparator<String>{  
    @Override  
    public int compare(String first, String second) {  
        return first.length()-second.length();  
    }  
}
```

这是一个泛型接口，其中的T代表某个具体的数据类型。

compare是Comparator<T>接口定义的方法，它的返回值决定了两个T类型的对象“谁大谁小”。

基于接口的“回调”实例-2

```
public class CallbackExample {  
    public static void main(String[] args) {  
        String[] friends={"Mary","John","Li","Peter"};  
        //sort方法内部会回调LengthComparator的compare方法  
        Arrays.sort(friends,new LengthComparator());  
        //输出按字符串长度排序的结果:  
        //Li,Mary,John,Peter,  
        for(var friend:friends){  
            System.out.print(friend+",");  
        }  
    }  
}
```

请查看一下Arrays.sort方法，看看其内部是怎样“回调”外部传入对象的compare()方法的。

CallbackExample.java

接口与抽象类的区别



抽象类是一个不完全的类，与普通类相比，不同之处在于它包容了抽象方法，并且不能直接实例化，除此之外，与普通类没大的分别。而接口只是表明类应该具有哪些“外部”特征或功能特性，不涉及具体的实现细节。



接口基本上不具备继承的任何具体特点，它仅仅承诺了外界能够调用的方法。



一个类一次可以实现若干个接口，但一个类只能继承一个父类。

面向对象建模训练



现实生活中随处可见各种树木，请任选两三种树木，用本讲所介绍的方法，为其建立一个面向对象的编程模型，并用Java实现之。

下一讲



- 抽象类与接口是面向对象编程语言中实现多态的主要方式。
- 在下一讲中，介绍初学者通常难以理解与把握的“多态”特性。
- 另一方面，接口又在函数式编程中起到了关键的作用，在后面介绍到“函数式编程”时，你对此会有深切的体会。